المطعم: أفكار إضافية

ليست واجباً · لمن يريد أن يتوسّع

هذه أفكار **لمن انتهى من الواجب** ويريد أن يتدرّب أكثر. لا يوجد حل جاهز لها هنا، بل **طريقة تفكير**: كيف يبدأ الطالب، وماذا يسأل نفسه، وأين يكتب. جرّب أن تحل الفكرة بنفسك بعد أن تقرأ طريقة التفكير.

1 كيف أفكر في أي فكرة جديدة؟

قبل أن تكتب أي سطر، اسأل نفسك هذه الأسئلة **بالترتيب**:

#	السؤال	ماذا يعني الجواب؟
1	**ماذا يتغير؟** ما الذي سيعمل بشكل جديد؟	نعرف أين نعدّل
2	**من يعرف المعلومة؟**	الصف الذي يعرفها هو الذي يحسب
3	**هل هو؟** هل هو نوع جديد من شيء موجود؟	وراثة
4	**هل يستطيع؟** هل هي قدرة تخص بعضهم فقط؟	واجهة
5	**من يبني؟ من يحمل؟ من يستعمل فقط؟**	تركيب أو تجميع أو ارتباط
6	**هل أحتاج أن أحمي قيمة؟**	Encapsulation: `private` و `set` يتحقق

القاعدة الذهبية: الصف الذي **يملك المعلومة** هو الذي يتصرف بها. لا نكتب المنطق في `main` إلا إن لم نجد له صفاً أنسب.

2 الأفكار

1 خصم الزبون المميز

⭐ سهل

الفكرة: إن كان الزبون `isVip()` يُخفَّض مجموع طلبه 10% قبل أن يدفع.

كيف نفكر؟

- ماذا يتغير؟ طريقة حساب مجموع الطلب.
- من يعرف أن الزبون مميز؟ `Customer` (عنده النقاط).
- من يعرف المجموع؟ `Order` (عنده العناصر).
- `Order` يحمل الزبون أيضاً (تجميع)، فهو يعرف **الاثنين** معاً.

المفهوم: المسؤولية: الصف الذي يعرف كل ما يلزم هو الذي يحسب.

أين نكتبه؟ في `Order` : تابع جديد مثل `totalAfterDiscount()` . لا نكتبه في `Customer` لأن الزبون لا يعرف المجموع، ولا في `main` لأننا لا نريد تكرار القاعدة.

تلميح: اسأل `customer.isVip()` داخل `Order` . جرّب زبوناً مميزاً وآخر عادياً.

2 بحث عن صنف بالاسم

⭐ سهل

الفكرة: بدل أن يكتب الزبون رقم الصنف، يكتب اسمه (مثل `Pizza`) ونجده.

كيف نفكر؟

- ماذا يتغير؟ طريقة اختيار الصنف في الطلب.
- ما الذي يرجعه البحث؟ `MenuItem`.
- ماذا لو لم نجد الصنف؟ نرجع `null` (لا شيء).
- ماذا يجب أن نفعل بعد البحث؟ **نتأكد أنه ليس** `null` قبل أن نستعمله، وإلا ينهار البرنامج.

المفهوم: التعامل مع `null` والتحقق قبل الاستعمال.

أين نكتبه؟ تابع `findItem(menu, name)` في `Main`، لأن القائمة موجودة هناك.

تلميح: قارن الاسم بـ `equals` وليس بـ `==`. وإن لم يجده اطبع "Not found" واطلب الاسم من جديد.

3 عدّاد الأطباق

⭐ سهل

الفكرة: كم قطعة في الطلب كله؟ (بيتزا × 2 + عصير × 1 = 3).

كيف نفكر؟

- من يعرف الكميات؟ كل `OrderItem`.
- من يعرف كل العناصر؟ `Order`.
- إذن `Order` يمر على عناصره ويجمع الكميات، كما فعل مع `total()`.

المفهوم: حلقة تجميع، وتقليد لتابع موجود.

أين نكتبه؟ في `Order`: `itemCount()`.

تلميح: انسخ شكل `total()` وغيّر ما يُجمّع: الكمية بدل المبلغ.

4 أكثر من طلب لأكثر من زبون

⭐⭐ متوسط

الفكرة: ثلاثة زبائن، لكل منهم طلب، والموصّل يوصّل الثلاثة بحلقة واحدة.

كيف نفكر؟

- أين نحفظ الطلبات؟ قائمة `ArrayList<Order>` في `main`.
- لماذا ليس داخل `Driver`؟ لأن الموصّل **لا يملك** الطلبات، هو يوصّلها فقط (ارتباط).
- كيف يوصّل؟ حلقة على القائمة: `driver.deliver(order.getCustomer(), order)`.
- لاحظ أننا نأخذ الزبون من الطلب نفسه (`getCustomer`) لأن الطلب يحمله.

المفهوم: الارتباط (وسيطان لتابع) مع حلقة، وتعدد الأشكال.

أين نكتبه؟ في `main`.

تلميح: ابنِ كل طلب بزبون مختلف، ولا تنسَ أن الزبون يُنشأ **قبل** الطلب (تجميع).

5 السعر وقت الطلب

⭐⭐⭐ صعب

الفكرة: غداً يغيّر صاحب المطعم سعر البيتزا. الطلبات القديمة يجب أن تبقى بالسعر الذي دُفع.

كيف نفكر؟

- ما المشكلة؟ `OrderItem` يحمل **إشارة** إلى الصنف، والسعر يُقرأ من الصنف الحالي. إن تغيّر السعر تغيّر مجموع طلب قديم!
- ما الذي يجب أن يبقى ثابتاً؟ السعر **لحظة الطلب**، وهو قرار انتهى.
- ما الحل؟ `OrderItem` **ينسخ** السعر في بنّائه، ويحتفظ به كخاصية خاصة به.
- ثم `subtotal()` يستعمل السعر المنسوخ وليس سعر الصنف.

المفهوم: الفرق بين **الإشارة** إلى شيء و**نسخة** منه. وهي فكرة تُستعمل في كل برامج الفواتير.

أين نكتبه؟ في `OrderItem` .

تلميح: اختبر: أنشئ طلباً، ثم غيّر سعر الصنف بـ `setPrice` ، ثم اطبع مجموع الطلب القديم. يجب ألا يتغير.

6 نوعان من الطلب: توصيل وداخل المطعم

⭐⭐⭐ صعب

الفكرة: طلب داخل المطعم له رقم طاولة، وطلب توصيل له رسوم توصيل.

كيف نفكر؟

- هل هما **نوعان من الطلب**؟ نعم ← `Order` يصبح **مجرداً**.
- ماذا يشتركان؟ الزبون والعناصر والمجموع ← عند الأب.
- ماذا يختلفان؟ رقم الطاولة (داخل المطعم) ورسوم التوصيل (توصيل) ← عند الأبناء.
- من يحتاج الموصّل؟ **طلب التوصيل فقط**. فـ `deliver` يجب أن يقبل `DeliveryOrder` .

المفهوم: الوراثة مع تعدد الأشكال، وصف مجرد جديد.

أين نكتبه؟ `DineInOrder` و `DeliveryOrder` يرثان من `Order` .

تلميح: اجعل `total()` في طلب التوصيل يضيف رسوم التوصيل. (وهنا يمكن أن نضيف فكرة حجز طاولة لاحقاً كنوع ثالث.)

7 صف للمطعم

⭐⭐ متوسط

الفكرة: صف `Restaurant` يحمل القائمة والموظفين.

كيف نفكر؟

- لماذا لم نضعه في التمرين؟ لأن القصة كانت تكفي بدونه، وأردنا أن تبقى بسيطة.
- متى نحتاجه؟ إن صار عندنا **أكثر من مطعم**، أو أردنا أن نقول "موظفو هذا المطعم".
- ما علاقته بالقائمة والموظفين؟ **تجميع**: الأصناف والموظفون يوجدون قبل المطعم ويبقون بعده.
- هل يبني الأصناف بنفسه؟ لا، تأتيه جاهزة (`addItem(menuItem)`).

المفهوم: التجميع، ومتى نضيف صفاً جديداً.

أين نكتبه؟ صف جديد `Restaurant` فيه `ArrayList<MenuItem>` و `ArrayList<Employee>` .

تلميح: ثم انقل حلقات القائمة والرواتب من `main` إليه، وانظر كيف يصير `main` أنظف.

8 الشخص زبون وموظف معاً

⭐⭐⭐ صعب

الفكرة: شيف يطلب غداءً من مطعمه. هو موظف **وزبون** في آن واحد. كيف نمثله؟

كيف نفكر؟

- الوراثة تقول "هو": هل هو زبون **أم** موظف؟ لا نستطيع أن نجعله الاثنين (الصف يرث أباً واحداً).
- ما الحل؟ نغيّر السؤال: الزبون والموظف **دوران** يقوم بهما الشخص، وليسا نوعين منه.
- إذن `Person` يحمل **أدواراً** (`Role`)، وكل دور له تصرفاته (تركيب أو تجميع بدل وراثة).
- وهذا يغيّر التصميم كله، ولا نحتاجه في تمريننا.

المفهوم: حدود الوراثة، وأن "هو" لا تصلح لما يتغير. فكرة متقدمة للنقاش فقط.

أين نكتبه؟ لا نكتب كوداً: ناقش الفكرة مع زملائك أو مع مدرّسك.

تلميح: اسأل: "ماذا لو انتقل الشخص من زبون إلى موظف؟" ستجد أن الوراثة لا تسمح بتغيير النوع بعد الإنشاء.

3 أي فكرة أبدأ بها؟

الفكرة	الصعوبة	تتدرب فيها على
1) خصم الزبون المميز	⭐ سهل	من يحسب؟ (المسؤولية)
2) بحث بالاسم	⭐ سهل	`null` والتحقق
3) عدّاد الأطباق	⭐ سهل	حلقة تجميع
4) أكثر من طلب	⭐⭐ متوسط	الارتباط وتعدد الأشكال
7) صف المطعم	⭐⭐ متوسط	التجميع
5) السعر وقت الطلب	⭐⭐⭐ صعب	إشارة مقابل نسخة
6) نوعا الطلب	⭐⭐⭐ صعب	وراثة جديدة
8) زبون وموظف معاً	⭐⭐⭐ صعب	نقاش فقط